

Configuration Management

Community Health Survey System

ANSIBLE	Ubuntu 26.04.1 LTS on WSL2
TARGET	Local Linux node, managed with Ansible's local connection
RESULT	Successful first run; second run changed zero resources

Evidence date: 8 October 2026 (Asia/Calcutta). This report records the actual Ansible executions and target verification. No Jenkins job was run.

1. Objective and approach

Identify CHSS deployment-host prerequisites and express the correct target state as a repeatable Ansible playbook. The project deploys CHSS as a Docker image. The host therefore needs Docker Engine; Java and Maven belong to the container build/runtime images, not the host installation.

Target-node environment

A usable Ubuntu WSL distribution was not present initially. WSL Ubuntu was installed non-interactively with `ws l --install --distribution Ubuntu --no-launch`. Ubuntu 26.04.1 LTS launched as a real WSL2 distribution with systemd as PID 1, Python 3, APT, and systemctl. Ansible Core 2.20.1 was installed from Ubuntu repositories. Ansible targets the same distro as localhost using a local connection; it is not a Docker container or Docker Desktop's internal distro.

The Ubuntu WSL target has its own Docker Engine 29.1.3. Docker Desktop's separate Engine 29.7.2 and pre-existing containers were left untouched. The playbook provisions host prerequisites only; it does not deploy the application.

2. Configuration specification

Requirement	Target state	Ansible resource / verification
OS and Ansible	Ubuntu 26.04.1 LTS; Ansible Core 2.20.1; Python 3	OS-family assert; fact gathering; syntax and inventory checks passed
Packages	ca-certificates, docker.io, python3 installed	APT state=present; first run changed packages, second run unchanged
Java / Maven	Java 17 JRE in image; Maven 3.9.9 in build stage	No host install; Dockerfile and pom.xml are source of truth
Users	No host app user; container runs as image user chss	No user task: duplicate host identity is unnecessary
Directories / files	No host app/config/log/data files required	No file/directory tasks; no credentials created
Ports	8081 container app; 18082 deployment host map	No host firewall or app deployment in this play
Other ports	8082 Jenkins, 8083 Tomcat are separate Windows services	Not configured on Ubuntu target
Database	Week 12 Docker run uses in-memory H2	No MySQL server/client or credentials on target
Services	Docker enabled and active under systemd	service module; checked systemctl active/enabled and Docker CLI

3. Packages and runtime boundary

Ubuntu host packages managed by the playbook are ca-certificates, docker.io, and python3. Python 3 was present before Ansible could run and remains declared to make the target package state explicit. The actual package versions after execution were docker.io 29.1.3-0ubuntu4.1, ca-certificates 20260601~26.04.1, and python3 3.14.3-0ubuntu2.

The CHSS Dockerfile uses Maven 3.9.9 with Temurin 17 to build the executable WAR, then Temurin 17 JRE to run it. No Java or Maven host install is required for this Docker deployment. Week 12 uses in-memory H2, so MySQL is not a prerequisite for this target.

4. Users, directories, and files

Ansible ran as root on localhost because package and service state require administrative access. The playbook does not create a second host account. The Docker image already defines an unprivileged system account named chss and owns /app inside the container. No host-side CHSS application, data, log, or configuration directories are used by the existing container deployment. The playbook creates no files and embeds no credentials.

5. Ports and services

TCP 8081 is the application's container port. Week 12 maps host TCP 18082 to container 8081. Jenkins uses 8082 and Tomcat uses 8083 on separate Windows services. This host-prerequisite play opens no ports and does not start CHSS or MySQL. It manages only Ubuntu's docker.service.

6. Inventory

```
[chss_servers]
localhost ansible_connection=local ansible_python_interpreter=/usr/bin/python3
```

The Ansible inventory parser resolved localhost in the chss_servers group. No SSH endpoint, placeholder address, or secret is required.

7. Playbook behavior

The play gathers facts and asserts the Debian family, installs the package list with ansible.builtin.apt, ensures docker.service is enabled and started with ansible.builtin.service, then checks the Engine using a read-only Docker version command and assertion. An optional URI task checks /login only if chss_verify_deployed_app=true; it is off by default because this playbook does not deploy the app. No handlers are needed because no file configuration changes trigger a service restart.

8. Real execution evidence

Environment setup and validation were performed in Ubuntu WSL. The Ansible output below is transcribed from the actual command responses. The first run emitted a facts deprecation warning; the playbook was corrected to use `ansible_facts.os_family` before the second run.

First successful execution

```
PLAY [Configure prerequisites on the CHSS Docker deployment host] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Require a Debian-family Linux target] *****
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to `True` is deprecated.
Use `ansible_facts["fact_name"]` (no `ansible_` prefix) instead.
ok: [localhost] => {"changed": false, "msg": "All assertions passed"}

TASK [Install Docker and host verification prerequisites] *****
changed: [localhost]

TASK [Enable and start Docker Engine] *****
ok: [localhost]

TASK [Read Docker Engine version] *****
ok: [localhost]

TASK [Confirm Docker Engine is available] *****
ok: [localhost] => {"changed": false, "msg": "All assertions passed"}

TASK [Check the CHSS deployment endpoint when an app is already deployed] *****
skipping: [localhost]

TASK [Report application endpoint verification mode] *****
ok: [localhost] => {"msg": "CHSS endpoint http://127.0.0.1:18082/login was not checked because no
application deployment was requested."}

PLAY RECAP *****
localhost : ok=7 changed=1 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

Idempotency execution after the fact-variable correction

```
TASK [Gathering Facts] *****
ok: [localhost]
TASK [Require a Debian-family Linux target] *****
ok: [localhost] => {"changed": false, "msg": "All assertions passed"}
TASK [Install Docker and host verification prerequisites] *****
ok: [localhost]
TASK [Enable and start Docker Engine] *****
ok: [localhost]
TASK [Read Docker Engine version] *****
ok: [localhost]
TASK [Confirm Docker Engine is available] *****
ok: [localhost] => {"changed": false, "msg": "All assertions passed"}
TASK [Check the CHSS deployment endpoint when an app is already deployed] *****
skipping: [localhost]
TASK [Report application endpoint verification mode] *****
ok: [localhost] => {"msg": "CHSS endpoint http://127.0.0.1:18082/login was not checked because no
application deployment was requested."}

PLAY RECAP *****
localhost : ok=7 changed=0 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

The first run made one package-state change. The subsequent run changed zero resources, with no failed or unreachable hosts. The second run therefore verifies idempotency for the configured target state.

Continuation audit (8 October 2026)

The existing Ubuntu 26.04.1 WSL2 target was confirmed as a normal distribution, separate from docker-desktop. Inventory graph and syntax validation passed again. Two additional actual playbook executions each reported

ok=7, changed=0, unreachable=0, failed=0, skipped=1. The Docker service remained active and enabled; the Ubuntu Docker Engine responded as Client=29.1.3 Server=29.1.3. Full continuation command output is preserved in community-health-survey/chss/ansible/execution-log.txt.

9. Final verification and limits

Ansible inventory parsing and playbook syntax validation passed. Docker reports active and enabled; its Ubuntu Engine responds as version 29.1.3. Package versions were queried with dpkg-query. No host chss user was created. The optional CHSS endpoint task was skipped because this playbook prepares prerequisites rather than launching a container. Java 17 runtime behavior remains documented by the existing project image and Week 11/12 evidence.

The execution log contains setup commands, validation results, both Ansible recaps, and post-run checks. No terminal screenshot was captured; the PDF uses the real command output transcript rather than a fabricated image. No Jenkins job was run, and existing Docker Desktop containers and unrelated working-tree changes were left alone.

10. Conclusion

Week 13 now has a runnable, idempotent Ansible implementation against a real Ubuntu WSL2 target. The configuration reflects the project's Docker deployment boundary: it installs and verifies the Docker host service while avoiding unneeded host Java, Maven, MySQL, app users, directories, files, or ports.